AquiIoT

# ESP32 do Setor

Firmware `esp32_setor.ino`

Este documento descreve o funcionamento do ESP32 instalado em cada setor da edificação. Ele cria a rede Wi-Fi local onde os chaveiros se conectam, registra a presença de cada chaveiro, lê os sensores do ambiente (luminosidade, som, temperatura e umidade) e envia tudo para o backend na Raspberry Pi pela internet.

Projeto Integrador V — Engenharia de Computação
UNIPAMPA — Campus Bagé

Documento gerado em 26/10/2026

# 1. Visão geral

O ESP32 do setor trabalha em dois modos de Wi-Fi ao mesmo tempo (**AP + STA**):

- **Ponto de acesso (AP)**: cria a rede `AquiIoT_Setor_01` (IP 192.168.4.1, canal 6, até 4 clientes). Os chaveiros se conectam nela e enviam `POST /presenca`.
- **Estação (STA)**: conecta em um Wi-Fi com internet (configurado em `segredos.h`) e envia os dados ao backend por HTTPS.

Se o `segredos.h` não existir ou estiver vazio, o ESP32 funciona apenas localmente: os chaveiros continuam sendo registrados e os dados podem ser consultados pelas rotas HTTP do próprio ESP32 e pelo Monitor Serial.

> **Atenção ao canal:** quando a estação (STA) está conectada, o rádio do ESP32 passa a usar o canal do roteador, e o AP acompanha esse canal. O valor `CANAL_WIFI = 6` só vale enquanto o STA não está conectado.

# 2. Hardware e ligações

| Componente | GPIO | Tipo | Observação |
|---|---|---|---|
| LDR (divisor de tensão) | 34 | ADC1, entrada analógica | 0 a 4095 (12 bits). Valores no limite geram aviso. |
| Microfone (saída AO) | 35 | ADC1, entrada analógica | Mede a variação (máx − mín) numa janela de 25 ms. |
| DHT11 (dados) | 32 | Digital | Temperatura (°C) e umidade (%). Biblioteca Adafruit DHT. |
| Buzzer | 25 | PWM 2 kHz (passivo) | Desabilitado no código (HABILITAR_BUZZER = false). |

Os pinos 34 e 35 são somente entrada e pertencem ao ADC1, que continua funcionando com o Wi-Fi ligado (o ADC2 não pode ser usado junto com o Wi-Fi).

# 3. Configuração

## 3.1 Arquivo `segredos.h` (não vai para o Git)

Copie `segredos.exemplo.h` para `segredos.h` e preencha:

```
const char* WIFI_STA_SSID  = "NOME_DA_REDE_COM_INTERNET";
const char* WIFI_STA_SENHA = "SENHA_DA_REDE";
const char* URL_BACKEND    = "https://raspberrypi.SEU_TAILNET.ts.net";  // sem barra no final
```

## 3.2 Constantes no início do `esp32_setor.ino`

| Constante | Valor atual | Função |
|---|---|---|
| ID_SETOR / NOME_SETOR | SETOR_01 / Setor 01 | Identificação do setor nas APIs e no backend. |
| ESP_ID | ESP32-01 | Identificação da placa. |
| NOME_REDE / SENHA_REDE | AquiloT_Setor_01 / 12345678 | Rede criada para os chaveiros (mín. 8 caracteres). |
| CHAVE_DISPOSITIVO | MUDE_ESTA_CHAVE_DOS_CHAVEIROS | Chave que os chaveiros enviam no cabeçalho X-Device-Key. Deve ser igual no chaveiro. |
| CHAVE_BUZZER | MUDE_ESTA_CHAVE_PRIVADA_DO_BUZZER | Chave exigida em X-API-Key para acionar o buzzer. |
| MAX_CHAVEIROS | 8 | Tamanho da tabela de chaveiros na memória. |
| MAX_CLIENTES_WIFI | 4 | Conexões simultâneas no AP. |

> As chaves `MUDE_ESTA_...` são valores de exemplo. Antes de uso real, troque por valores próprios (no ESP32 e em todos os chaveiros).

## 3.3 Temporizações

| Constante | Valor | Significado |
|---|---|---|
| INTERVALO_SENSORES_MS | 5 s | Intervalo entre leituras dos sensores. |
| JANELA_MIC_MS | 25 ms | Janela de amostragem do microfone. |
| PRESENCA_EXPIRA_MS | 15 s | Sem mensagem nesse tempo, o chaveiro deixa de contar como presente. |
| INTERVALO_VERIFICAR_AP_MS | 10 s | Verificação/reinício do ponto de acesso. |
| INTERVALO_ENVIO_SENSORES_MS | 30 s | Envio das leituras ao backend. |
| INTERVALO_REENVIO_PRESENCA_MS | 60 s | Reenvio da presença de um chaveiro que continua no setor (o backend marca offline após 120 s). |
| ESPERA_APOS_FALHA_MS | 10 s | Pausa nos envios após falha de rede ou erro 5xx. |
| INTERVALO_RECONECTAR_STA_MS | 30 s | Nova tentativa de conectar ao Wi-Fi com internet. |
| TIMEOUT_BACKEND_MS | 5 s | Tempo máximo de cada requisição HTTPS. |
| TEMPO_BIP_MS / INTERVALO_MINIMO_BIP_MS | 250 ms / 1 s | Duração do bip e intervalo mínimo entre bips. |

## 4. Inicialização — setup()

Ao ligar, o ESP32 prepara os sensores, liga o ponto de acesso, tenta conectar à internet (se houver configuração), registra as rotas HTTP e faz a primeira leitura dos sensores.

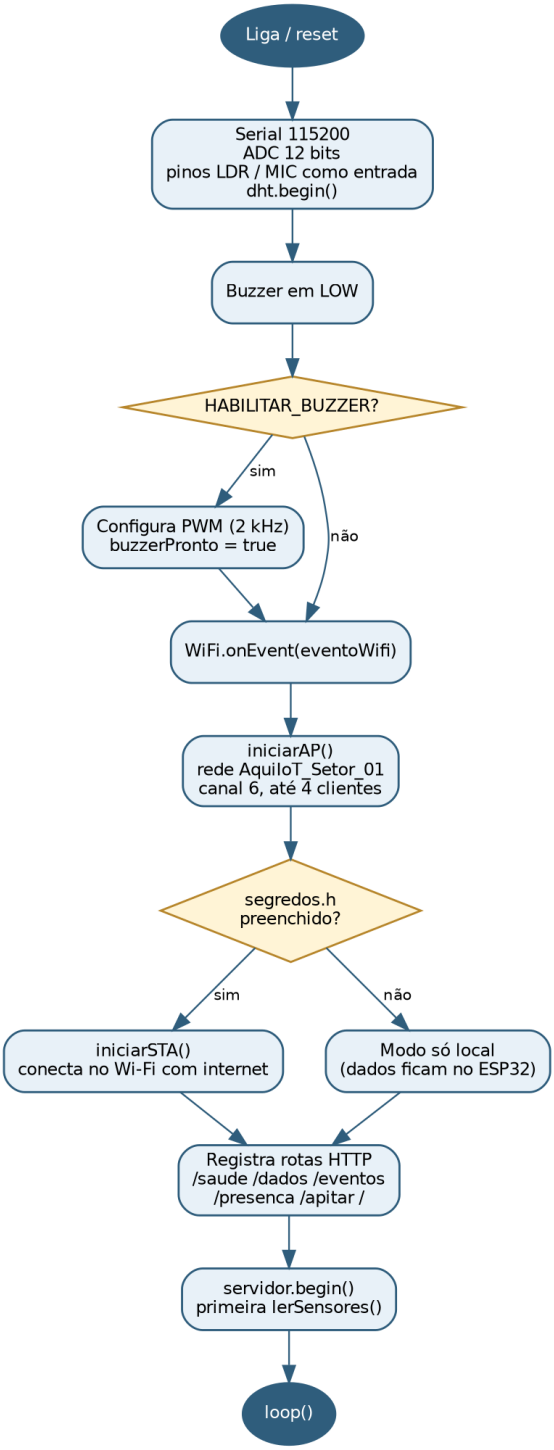



*Figura 1 — Fluxograma do setup()*

## 5. Laço principal — loop()

O `loop()` não usa `delay()` longos: cada tarefa verifica se já está na hora de agir, usando `millis()`. Assim o servidor HTTP continua respondendo aos chaveiros enquanto o resto acontece.

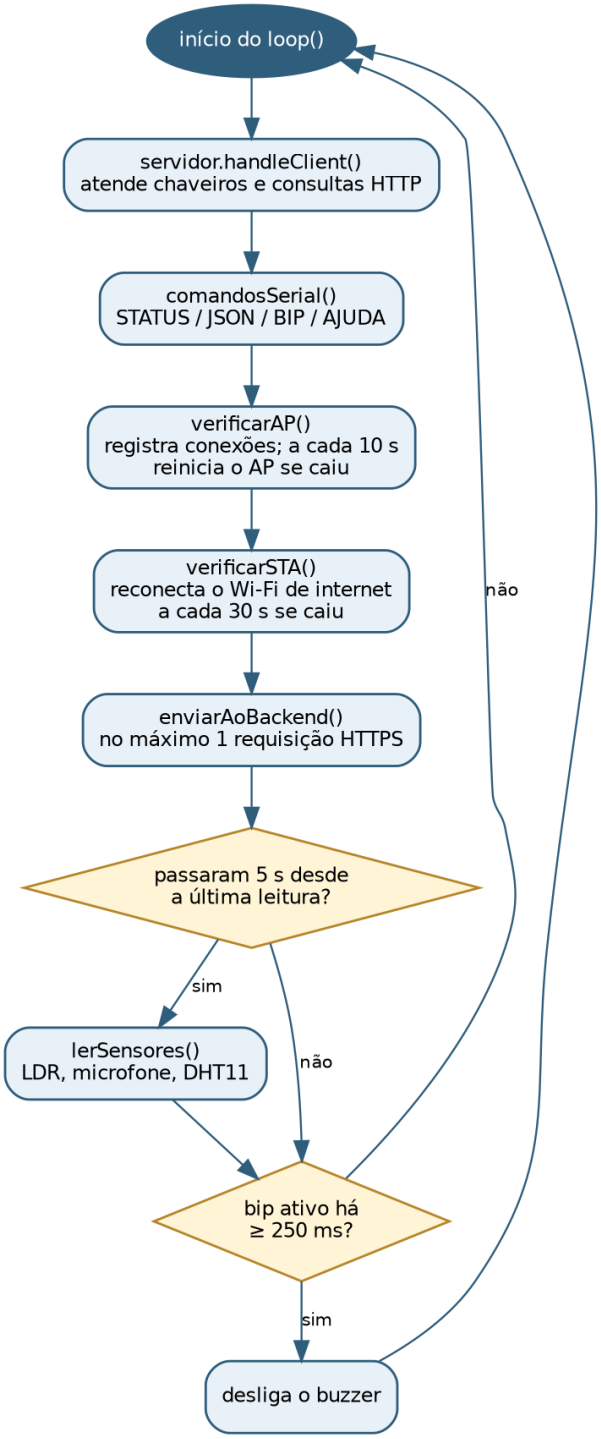



*Figura 2 — Fluxograma do loop()*

# 6. Registro de presença dos chaveiros

Cada chaveiro conectado ao AP envia, a cada 5 s, um POST `/presenca` com o seu ID e o RSSI (força do sinal que ele recebe do setor). O ESP32 valida a mensagem e atualiza a tabela em memória (até 8 registros).

```
POST /presenca HTTP/1.1
Host: 192.168.4.1
X-Device-Key: MUDE_ESTA_CHAVE_DOS_CHAVEIROS
Content-Type: application/x-www-form-urlencoded

id=00000001&rssi=-25
```

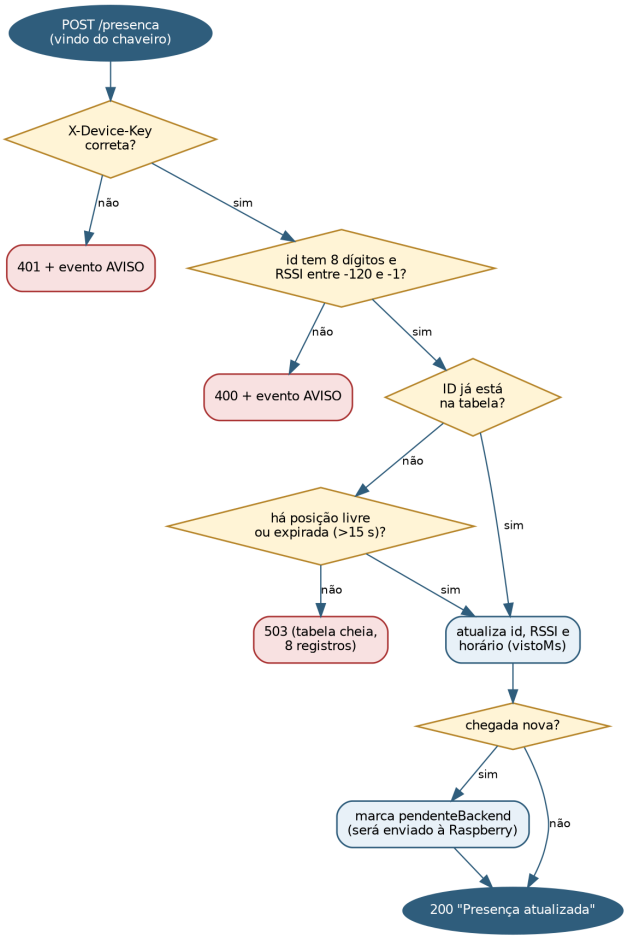



*Figura 3 — Tratamento de POST /presenca*

| Resposta | Quando acontece |
|---|---|
| 200 Presença atualizada | Mensagem aceita. |
| 400 | ID não tem exatamente 8 dígitos, ou RSSI fora de -120 a -1 (ou com caracteres inválidos). |
| 401 | Cabeçalho X-Device-Key ausente ou diferente de CHAVE_DISPOSITIVO. |
| 503 | Todas as 8 posições estão ocupadas por chaveiros ainda presentes. |

# 7. Leitura e diagnóstico dos sensores

A cada 5 s a função `lerSensores()` lê os três sensores. Para evitar alarmes falsos, uma condição só vira alerta depois de **3 leituras seguidas** (`LIMITE_FALHAS_SEGUIDAS`), e um evento de *RECUPERADO* é registrado quando ela some.

| Sensor | Condição suspeita | Evento gerado |
|---|---|---|
| LDR | ADC ≤ 2 ou ≥ 4093 (limite do conversor) | AVISO LDR: leituras no limite do ADC. Pode ser luz extrema ou divisor mal ligado. |
| Microfone | Mínimo ≥ 4090 ou máximo ≤ 5, com variação ≤ 2 na janela de 25 ms | AVISO MICROFONE: saída analógica persistente em nível extremo. |
| DHT11 | Leitura NaN ou umidade fora de 0–100 % | ERRO DHT11 após 3 falhas seguidas. A leitura inválida nunca é enviada como valor atual (vira null). |

Os sensores ficam **DEGRADADO** quando há alerta no DHT11 ou no microfone. O aviso do LDR é só diagnóstico e não deixa o setor degradado.

## 7.1 Histórico de eventos

Os eventos (INFO, AVISO, ERRO, RECUPERADO) ficam num buffer circular de 12 posições, somente em RAM: reiniciar o ESP32 apaga o histórico. Eles podem ser lidos em `GET /eventos` e são enviados ao backend um a um. Se mais de 12 eventos acontecerem antes do envio, os mais antigos são perdidos.

# 8. Envio ao backend (Raspberry Pi)

A função `enviarAoBackend()` faz **no máximo uma requisição por volta do loop**, para não travar o atendimento aos chaveiros (cada requisição HTTPS leva 1–2 s por causa do TLS). A ordem de prioridade é: registrar o setor → presença → sensores → eventos.

> A conexão HTTPS usa `setInsecure()`: o certificado do servidor não é conferido. Para produção, troque por `setCACert()` com a raiz ISRG Root X1 (Let's Encrypt).

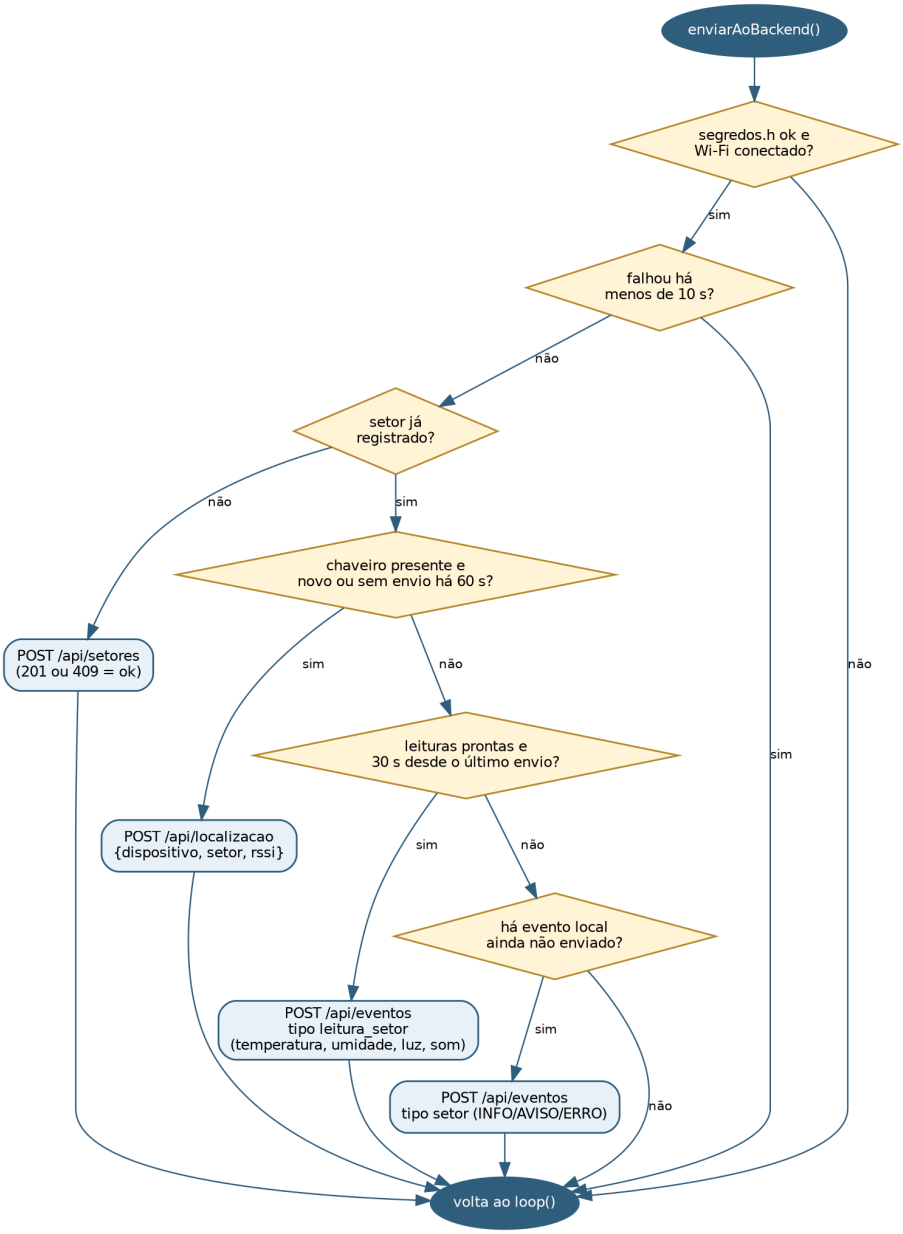



*Figura 4 — Prioridade dos envios ao backend*

| Rota no backend | Corpo JSON (exemplo) |
|---|---|
| POST /api/setores | {"nome":"Setor 01","esp32":"ESP32-01"} |
| POST /api/localizacao | {"dispositivo":"00000001","setor":"Setor 01","rssi":-25} |
| POST /api/eventos (leitura) | {"tipo":"leitura_setor","nivel":"info","setor":"Setor 01","mensagem":"Leitura dos sensores do setor.","detalhes":{"esp32":"ESP32-01","temperatura_c":22.2,"umidade_percentual":64.0,"luminosidade_adc":0,"som_variacao":0,"sensores_degradados":false}} |
| POST /api/eventos (evento) | {"tipo":"setor","nivel":"aviso","setor":"Setor 01","mensagem":"LDR: ...","detalhes":{"esp32":"ESP32-01","tipo":"AVISO","origem":"LDR","uptime_ms":12345}} |

Interpretação das respostas: **2xx** = aceito; **4xx** = dado recusado (não adianta reenviar, é dado como concluído; 409 no registro do setor significa “já cadastrado”); **5xx ou erro de rede** = falha, nova tentativa após 10 s.

## 9. Rotas HTTP do ESP32 (rede local)

| Rota | Método | Descrição |
|---|---|---|
| / | GET | Texto simples com a lista de APIs. |
| /saude | GET | Estado do setor: uptime, clientes Wi-Fi, conexão com backend, estado dos sensores. |
| /dados | GET | Leituras completas dos sensores e lista de chaveiros presentes com RSSI. |
| /eventos | GET | Histórico de eventos em memória (até 12). |
| /presenca | POST | Usado pelos chaveiros (X-Device-Key, id, rssi). |
| /apitar | POST | Aciona o buzzer por 250 ms. Exige X-API-Key. Responde 503 se o buzzer estiver desabilitado e 429 se pedido em menos de 1 s. |

## 10. Monitor Serial (115200 baud, final de linha “Nova linha”)

| Comando | Resposta |
|---|---|
| STATUS | Resumo do setor, sensores e diagnóstico. |
| JSON | Leitura atual em uma linha JSON. |
| BIP | Bip local (se o buzzer estiver habilitado). |
| AJUDA | Lista de comandos. |

Exemplo de saída real (26/10/2026):

```
=== AQUIIOT | SETOR 01 | TESTE LOCAL ===
[WIFI] Ponto de acesso em 192.168.4.1
[BACKEND] Conectando a rede a22...
[SENSORES] LDR: 0 | MIC: 4095 | Variacao: 0 | DHT11: 22.2 C / 64.0 % | Clientes Wi-Fi: 0 | ...
[BACKEND] Wi-Fi conectado. IP: 192.168.0.117
[BACKEND] Setor Setor 01 pronto no backend (HTTP 201).
[PRESENCA] Chaveiro 00000001, RSSI informado -25 dBm
[BACKEND] Presenca de 00000001 enviada.
```

Em placas com conversor USB CH9102 (como a usada nos testes), abrir o monitor serial pode manter o ESP32 em reset e a tela fica vazia. Se acontecer, pressione o botão EN/RST com o monitor aberto.

## 11. Compilação e gravação

- Placa: **ESP32 Dev Module** (FQBN `esp32:esp32:esp32`), núcleo esp32 3.x.
- Biblioteca necessária: **DHT sensor library** (Adafruit) e sua dependência Adafruit Unified Sensor.
- A pasta do sketch deve se chamar `esp32_setor` (mesmo nome do .ino).

```
arduino-cli lib install "DHT sensor library"
arduino-cli compile --fqbn esp32:esp32:esp32 -u -p /dev/ttyACM0 esp32_setor
```

## 12. Pontos de atenção conhecidos

- Nos testes, o LDR leu sempre 0 e o microfone sempre 4095: conferir as ligações dos GPIO 34 e 35.
- O buzzer está desabilitado (`HABILITAR_BUZZER = false`) até o circuito ser confirmado.

- Chaves de autenticação ainda com valores de exemplo.
- Certificado HTTPS não verificado (`setInsecure()`).
- Histórico de eventos apenas em RAM (12 posições).